

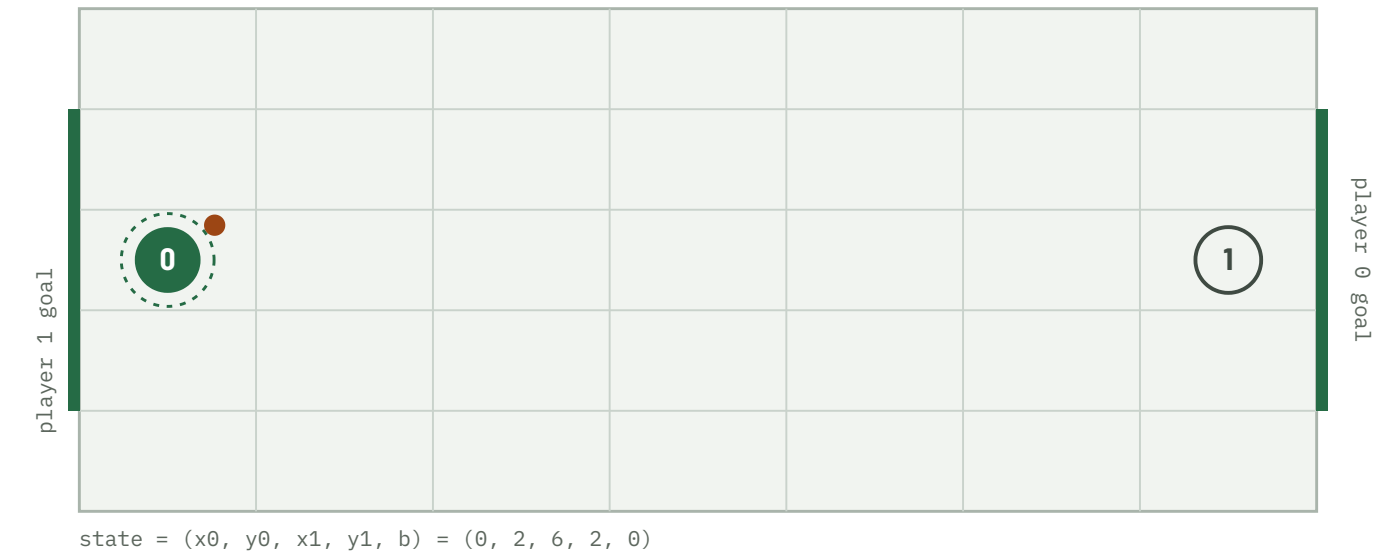
Pure-First Nash Q for Soccer

Where a two-player soccer Markov game does — and does not — need mixed strategies, and how much matrix-game work that saves.

FOUR QUESTIONS

RQ1 Existence — under what transition structures does a pure equilibrium exist at every state? **RQ2 Efficiency** — how much work does checking for a pure saddle first eliminate? **RQ3 Mechanism** — which spatial configurations force mixing? **RQ4 Robustness** — how do discounting and tolerance affect the split?

repo · soccer-markov-nash | 150+ tests | tables from experiments/*.csv | scope in docs/assumptions.md



The kickoff. 7x5 grid, 2380 non-terminal states. Player 0 carries the ball and attacks the right edge; player 1 the left. Actions U D L R are chosen simultaneously; reward is +1 / -1 / 0, zero-sum.

ABSTRACT

- 1** For the implemented deterministic A10 model, every one of the 2380 converged stage games has a **pure saddle**. Playing a pure saddle action everywhere is then a stationary pure-strategy equilibrium, found in 9 sweeps with no linear program.

- 2** A **coin-flip tie-break does not change this**; Littman's random *move order* does — but only when the goal mouth is more than one cell wide, and even then on ~4% of states (this configuration).

- 3** Those mixed states are **geometrically localised**: a decision tree predicts them from the state with precision 0.96, and 68 of 94 reduce to a 2×2 matching-pennies support.

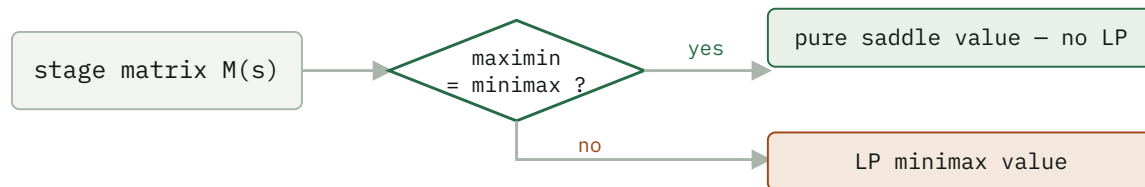
- 4** The **pure-first hybrid solver** matches the all-LP value function to 4e-16 while doing ~25× less matrix-game work; mirror symmetry halves it again.

Method

The game is zero-sum, so at every state the stage game is a payoff matrix $M(s) [a_0, a_1] = E[R + \gamma V(s')]$ and the Shapley (1953) fixed point is the minimax value:

$$\begin{aligned} V(s) &= \text{val}(M(s)) = \max_n \min_{k_3} (p^T M) [a_1] \\ &= \min_p \max_{a_0} (Mq) [a_0] \end{aligned}$$

The $Q = R + \beta \text{Nash}(Q')$ update is this operator. Three stage backups: **pure** (the maximin security value — a fast lower bound, not a Nash value when it is below the minimax), **mixed** (the LP minimax value, via `scipy HiGS`), and **hybrid** (the pure saddle value where maximin = minimax, the LP elsewhere — exact everywhere). Support enumeration (`support_enum.py`) finds *all* equilibria, including of general-sum games.



The pure-first hybrid. On the deterministic game the "yes" branch is taken at every one of the 2380 states, so the LP is never called.

RQ1 – Existence

MOVE ORDER	STOCHASTIC	CONVERGED STAGE GAMES WITH NO PURE SADDLE	STATIONARY PURE EQ?
deterministic (exact A10)	no	0 / 2380	yes (Claim B)
coinflip tie-break	yes	0 / 2380	yes (Claim B)
random move order	yes	94 / 2380 (this config)	no

IT IS THE MOVE ORDER, NOT THE RANDOMNESS

The deterministic and coin-flip value functions are *identical* ($\max |V_{\text{det}} - V_{\text{coin}}| = 0$ at every γ): optimal play never enters a losing contest, so the coin is never flipped. Only Littman's random move order — where a ball-steal depends on who resolves first *and* on both players' targets — creates matching-pennies subgames.

RQ2 – Efficiency

RANDOM MOVE ORDER, $\Gamma = 0.9$ – THE CASE WHERE THE LP IS ACTUALLY NEEDED

SOLVER	SWEEPS	V(KICKOFF)	MATRIX-GAME SOLVES	WALL CLOCK
pure (lower bound)	18	+0.000	0	<1 s
hybrid exact	108	+0.150	~9 700	~14 s
mixed exact	108	+0.150	~3.6 M	~8 min

The hybrid solves an LP on only the ~4% of states lacking a pure saddle — ~25× fewer — and matches the all-LP value to 4e-16. **Mirror symmetry** (`run_symmetric`): the game is anti-symmetric under board-flip + player-swap and every state has a distinct mirror, so the 2380 states are 1190 pairs — solve one, reconstruct the other by $V(\text{mirror}(s)) = -V(s)$.

Freeze-then-iterate, and its staleness

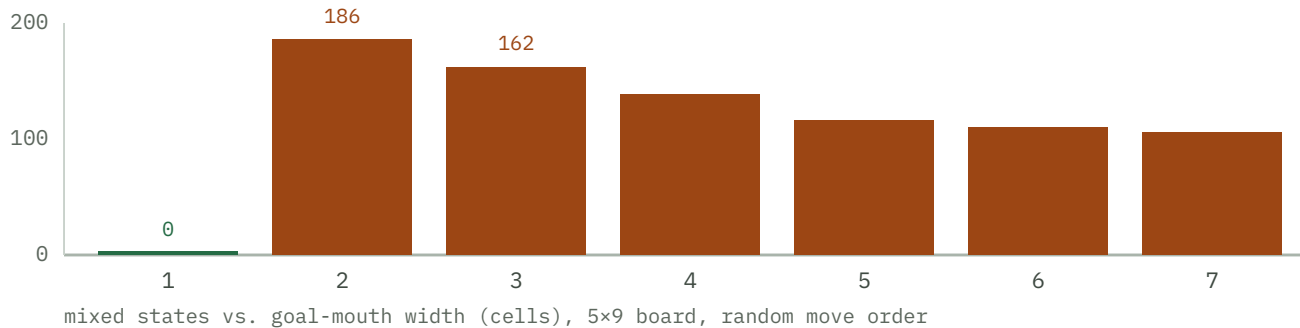
RANDOM MOVE ORDER, HYBRID SOLVER – SAME FIXED POINT, $|\text{DIFF}| < 3\text{E-}9$

	ROUNDS / SWEEPS	MATRIX-GAME SOLVES	WALL CLOCK
value iteration	108	9 672	13.8 s
policy iteration	21	1 879	17.2 s

5× fewer LP solves — but the frozen strategies' distance to the current Nash stays *maximal* (a full pure-strategy flip) for **16 outer rounds**, then collapses to $< 1\text{e-}8$. The thrashing eats the saving; on the deterministic game it is strictly worse. Value iteration with the per-sweep Nash cache wins here.

RQ3 – Mechanism

A stage game needs mixed strategies only when the goal mouth is more than one cell wide.



The gate. One-cell goal → the carrier has one threat, the defender one response, best responses stay pure. Two or more → the defender must guess. Every height-3 board (width 3–11) has zero mixed states.

Geometry predicts the label (scripts/geometry_model.py): a depth-4 decision tree on carrier-frame features separates mixed from the rest with **precision 0.96, recall 0.91**. The dominant feature is defender_can_intercept — the defender within one move of the carrier's forward cell ($P(\text{mixed}) = 0.44$ vs 0.002).

THE 94 MIXED STATES, BY EQUILIBRIUM SUPPORT × DEFENDER GEOMETRY

SUPPORT	DEFENDER	COUNT
2×2 (matching pennies)	ahead of the carrier	40
2×2	directly ahead, same row	24
2×1 / 1×2	directly ahead, same row	12
3×3 / 2×3 / 3×2	ahead	12
2×2	merely adjacent	4
2×1 / 1×2	ahead	2

~88 of 94 have the defender ahead; 68 are a 2×2 matching-pennies mix of carrier {advance, hold} against defender {block, intercept}. All 94 share one value across every equilibrium; 64 have a unique one.

An argument, not a proof. With one goal cell the value collapses to a one-dimensional race to that cell; each player then has a best-response direction independent of the other, so the stage game has a pure saddle. 19/19

single-cell configurations (boards 3×3 to 11×5, γ 0.5 to 0.99) show zero mixed states. Sketch in docs/mechanism.md; formalising it is open.

RQ4 – Numerical robustness

The value is three numbers

value_bracket returns what the row player guarantees ($\min_k (pM)_k$), gets ($p^T Mq$), and can reach ($\max_i (Mq)_i$). The true value is bracketed, so the width certifies the LP error.

FINDING

With scipy's HiGHS the three agree to 1.1e-16 per stage game, 8.7e-10 accumulated. The concern is real for older vertex/simplex solvers — a non-issue here.

Discounting and tolerance

	DETERMINISTIC	RANDOM (7×5)
mixed states, $\gamma = 0.5 \rightarrow 0.9 \rightarrow 0.995$	0 → 0 → 0	122 → 94 → 102
classification split, rel_tol 1e-12 → 1e-3	stable	604 / 1682 / 94 (stable)
classification, rel_tol 1e-2 / 1e-1	–	80 / 0 mixed
fixed 0.1-rounding flips a saddle	0	62

Discounting mildly shifts which states mix; the classification is otherwise robust across nine orders of magnitude of tolerance, breaking only as the tolerance approaches the real minimax – maximin gaps. Fixed 0.1-rounding — proposed to force indifference — misclassifies 62 states, exactly the small-entry mixed region.

Limitations

- The A10 page redacts the ID-specific start position and goal rows; this repo uses the standard Littman geometry (configurable). Qualitative results hold across 40+ board configurations; the exact 3.9% is the 7×5 / 3-cell-goal case.
- Several collision sub-cases (carrier vs. stationary opponent, bump = steal, swap possession) are *interpreted* from the two stated A10 rules, not quoted. If course staff intended otherwise, Claim A must be re-measured. See docs/assumptions.md.
- **Claim A** (every converged stage game has a pure saddle) and **Claim B** (a stationary pure equilibrium exists) are established by enumeration. **Claim C** (the theoretical game necessarily has one, over all settings) is *not* — that needs a proof.
- random and coinflip are different game definitions, isolating the collision rule and the tie-break respectively — not the A10 game.

Secondary validation — the policy plays correctly

- Duality gap $V0_{br}(s_0) + V1_{br}(s_0) < 1e-9$ for every move order — no opponent beats the game value.
- Nash vs. Nash reproduces the value: forced draws in the deterministic and coin-flip games; +0.157 empirical discounted return (vs. +0.150 computed) in the random game.
- A best response to one assumed opponent is fragile: the Part 2 policy has exploitability 0.43 — worse than random — which is the A10 competition's warning about non-equilibrium submissions.

Beyond zero-sum, and open questions

- **General-sum** — **done**. `markov_game.py` solves 2-player general-sum Markov games by enumerating *all* stage equilibria (`support_enum.py`) and selecting one — the notes' "largest sum of values", a no-op for zero-sum but decisive for Battle of the Sexes (it avoids the value-destroying mixed equilibrium). Pure-first throughout.
- **The dog / debate game** — **done**. The group's second game (`dog_game.py`) has a closed-form Nash — both players in their corners, dog at $w_a \cdot \text{house}_a + w_b \cdot \text{house}_b$ — and the general-sum solver hits it exactly in 1D and 2D. Finding: *every* stage game of the discretised dog game has a pure Nash equilibrium, so pure-first pays off here too.
- **Function approximation** — **partial**. A network fit to the stage matrices (`nash_dqn.py`, $5 \rightarrow 96 \rightarrow 96 \rightarrow 16$ with biases, 500 epochs) trails the exact solver badly:

	EXACT HYBRID NASH-Q	NEURAL NASH-Q
wall clock	9 sweeps, 0.45 s	500 epochs, 55 s
max $ V - V_{\text{exact}} $	0	0.54
action agreement	100%	53%
exploitability	0	0.43

The network fits the ~ 1000 zero-value states but misses the γ^k structure — about as exploitable as random play. Keep the exact solver as ground truth.

- **A proof (open)**. Can "one-cell goal $\Rightarrow$ pure everywhere" be turned into a theorem about the transition structure?
- **Continuous actions (open)**. The dog game's real form has a continuous action space; projected gradient on the neural Q is next, with the discrete solver as the reference.

soccer_nash — `game.py` · `matrix_games.py` · `support_enum.py` · `nash_q.py` · `markov_game.py` ·
`dog_game.py` · `numerics.py` · `geometry.py` · `symmetry.py` · `tree.py` · `best_response.py` ·
`exploit.py` · `mlp.py` · `nash_dqn.py` · `opponents.py` · `shaping.py` · `simulate.py` ·
`experiment.py`
scripts — `experiments.py` · `analyze.py` · `numerics.py` · `equilibria.py` · `geometry_model.py` ·
`policy_iteration.py` · `selfplay.py` · `nash_dqn.py` · `dog_game.py` · `a10_part1.py` ·
`a10_part2.py` · `a10_competition.py`
experiments — `baseline.csv` · `gamma_sweep.csv` · `tolerance_sweep.csv` · `board_sweep.csv` ·

goal_mouth_sweep.csv · one_cell_goal.csv

env: 7×5 grid, 2380 states, zero-sum ± 1 , $\gamma = 0.9$